

MPU-6050 系列

生产级 IIO 内核驱动

项目说明书 · Project Manual

Linux 内核驱动

IIO 子系统

STM32MP157

嵌入式 / BSP

项目名称	mpu6050-iio(MPU-6050/6500/9250 IIO 驱动)
仓库	github.com/fyy0619-cell/xiangmu
作者	fyy0619-cell
目标平台	STM32MP157 PRO(ARM Cortex-A7,Linux)
许可	GPL-2.0-only
日期	2026-09-22

一、项目背景

1.1 问题出发点

网络上绝大多数 MPU-6050 教程止步于"用 I2C 读 14 字节寄存器并 printf 打印",既不挂接内核子系统框架,也不涉及功耗管理、并发保护、事件上报、测试与文档等真实工程要素。这类"例程式"代码在求职面试中无法体现工程深度,且高度雷同,难以证明候选人具备量产驱动的开发能力。

1.2 项目目标

做一个**形态上等同于企业实际交付的 vendor BSP 驱动**的项目:芯片本身(MPU-6050)虽普通,但把驱动的工程完成度拉到量产水准——多芯片变体抽象、硬件 FIFO 批量搬运、运行时电源管理、运动检测事件、出厂自检、完整设备树绑定与 ABI 文档、持续集成(CI)与测试。差异化不在芯片,而在工程。

1.3 框架选型理由

选择挂接 Linux IIO(Industrial I/O)子系统,而非直接实现裸字符设备。原因:

- IIO 与相机领域的 **V4L2 同源**——都是"统一设备模型 + 标准用户空间 ABI + 可选缓冲/硬件触发数据流"的框架思想,但复杂度对入门者更友好。
- 手机 SoC 的传感器子系统正是基于 IIO,方向与高通等 SoC 厂商的岗位高度对口。
- 能完整练到"设备模型 / 通道 / 触发 / 缓冲 / 事件 / 电源"等一整套内核框架能力。

1.4 面向岗位

本项目对应的产业链岗位包括:传感器原厂(TDK InvenSense、Bosch Sensortec、ST)的驱动/应用工程师;SoC 平台厂(高通、联发科、展锐、瑞芯微、NXP)的 sensor 子系统 / BSP 工程师;终端设备厂(手机、可穿戴、无人机、机器人、AR/VR)的内核/驱动集成岗。技能栈是"嵌入式 Linux 驱动 / BSP 工程师"的通用底座。

二、技术栈

2.1 内核侧(驱动本体)

类别	采用技术 / 子系统	作用
目标框架	Linux IIO 子系统	通道模型、sysfs/字符设备 ABI、buffer、trigger、events
寄存器抽象	regmap(regmap-i2c)	统一读写、缓存、debugfs dump、总线无关
总线	I2C(i2c_driver)	与 MPU-6050 通信;预留 SPI 前端扩展
设备匹配	Device Tree + OF match	compatible = "invensense,mpu6050"
数据搬运	triggered buffer(kfifo)+ 硬件 FIFO 水位	中断驱动的批量采集
中断	GPIO threaded IRQ + iio_trigger	DRDY 数据就绪 / FIFO 溢出 / 运动检测
电源管理	runtime PM + system suspend/resume	autosuspend 到 sleep,省电
事件	IIO events	wake-on-motion 阈值中断
资源管理	devm_ 全托管	无手写 remove,错误路径用 dev_err_probe
并发	mutex + claim_direct_mode	配置写 / 数据读 / 缓冲互斥

2.2 工程配套

类别	技术
构建	Kbuild(双用途 Makefile:外部模块 + 交叉编译)、Kconfig
交叉编译	arm-linux-gnueabi- 工具链,在 Ubuntu 上 build
文档	DT binding(YAML,可过 dt_binding_check)、sysfs ABI 文档、架构说明
CI	GitHub Actions:拉取 Linux 6.1/6.6 源码 → 配 IIO 依赖 → 编译模块;checkpatch;userspace 构建
代码规范	Linux kernel coding style、SPDX 许可标识、checkpatch
验证	C 测试程序(sysfs 换算校验重力、缓冲流式读取)、上板冒烟脚本

2.3 硬件与联调环境

- 板卡:100ASK STM32MP157 PRO(Cortex-A7 + M4,跑完整 Linux)
- 传感器:MPU-6050(I2C 地址 0x68,带 FIFO 与 DRDY/motion 中断)
- 联调:板 ↔ Ubuntu VM 网口直连(TFTP/NFS 加载模块),串口 CH340 @115200 看 dmesg

三、项目流程

3.1 开发流程(工程方法)

项目采用"设计先行 + 分里程碑"的工程流程,留有完整过程记录(docs/superpowers/):

- 1. **需求与设计(spec)**:明确框架选型(IIO)、硬件(STM32MP157 + MPU-6050)、范围(生产级),产出设计文档。
- 2. **实现计划(plan)**:拆成 M1–M4 里程碑,每步含代码、验收命令与预期输出。
- 3. **编码实现**:按 mainline vendor 驱动方式多文件拆分实现。
- 4. **文档与 CI**:补 DT binding、ABI、架构说明,配 GitHub Actions 交叉编译。
- 5. **硬件 bring-up**:按 M1→M4 在真板逐步点亮验证。

3.2 里程碑

里程碑	内容	验收
M1 板级 bring-up	设备树节点 + i2c probe + WHO_AM_I + regmap	dmesg 见设备,i2cdetect 0x68
M2 raw 通道 + 量程/采样率	read/write_raw,scale/offset/ODR,available 列表	静止合加速度 ≈ 9.8 ,iio_info 干净
M3 触发缓冲	triggered buffer,软件触发先跑通	iio_readdev 出数据
M4 硬件触发 + 进阶	DRDY 中断触发、FIFO 水位、runtime PM、events、self-test	100Hz 自动流、事件、自检 pass

3.3 运行时数据流

路径一 • sysfs 单次(direct mode):

```
cat in_accel_x_raw
→ read_raw(RAW)
→ iio_device_claim_direct_mode()    # 与缓冲互斥
→ pm_runtime_resume_and_get()      # 唤醒芯片
→ regmap_bulk_read(ACCEL_XOUT_H)   # 16-bit 大端
→ 换算返回
```

路径二 • 缓冲流式(triggered buffer):

```
选通道 scan_elements/*_en → buffer/enable=1
→ DRDY / FIFO 中断(GPIO IRQ, threaded)
```

- `iio_trigger_poll_nested()`
- `pollfunc`: 单帧读 或 FIFO 批量 drain
- `iio_push_to_buffers_with_timestamp()`
- 用户 `read(/dev/iio:deviceN)`

四、项目难点

难点	问题	解决思路
框架抽象与分层	如何做到总线无关、可扩展 SPI,而非把 I2C 逻辑写死	仿 mainline inv_mpu6050,拆成 core + i2c 两个模块;core 只依赖 regmap 与 iio,总线前端通过 mpu6050_core_probe() 注入
缓冲 + 硬件触发	如何把"硬件中断 → 批量搬运 → 用户读"这条数据流正确接起来,并与 sysfs 单次读互斥	triggered buffer + iio_trigger;threaded IRQ 中读 INT_STATUS 后 iio_trigger_poll_nested;单次读用 claim_direct_mode 互斥
硬件 FIFO 水位	每样本一次中断在高 ODR 下功耗/CPU 占用高;需批量搬运	watermark>1 时启用片上 FIFO,按 FIFO_COUNT 逐 14 字节帧 drain;通过标准 hwfifo ABI 暴露
电源管理	空闲省电与随取随用的平衡;system suspend 与 runtime 协同	runtime PM autosuspend 到 sleep;单次读/缓冲开启前 resume_and_get;system sleep 走 pm_runtime_force_suspend/resume
自检的整数实现	内核禁用浮点,datasheet 的 factory-trim 是浮点多项式	测量自检响应(actuator 开-关之差),用整数绝对窗口判定 pass/fail;浮点 trim 作为后续扩展并诚实标注
并发正确性	配置写、单次读、缓冲搬运、事件可能并发访问寄存器	st->lock 串行化;缓冲期间拒绝 direct-mode 读;中断用 threaded+ONESHOT 保证可安全做 I2C
跨内核版本兼容	目标板 ST 5.4 BSP 缺少较新 API	基线定为 5.15/6.1;为 5.4 提供 pm_runtime_resume_and_get 等最小兼容 shim;CI 对 6.1/6.6 双版本编译
数据大小端与打包	MPU 输出为 16-bit 大端,缓冲扫描需按 scan_index 对齐	scan_type 声明 IIO_BE/16bit;寄存器块(accel/temp/gyro)顺序与 enum 一致,整块读入即为合法 scan payload

五、项目细节

5.1 代码组织

drivers/iio/imu/mpu6050/	
mpu6050.h	寄存器映射、chip_info、状态结构、函数原型
mpu6050_core.c	设备模型 / 通道 / read write_raw / PM / probe
mpu6050_i2c.c	I2C 前端(模块入口、变体匹配)
mpu6050_buffer.c	triggered buffer + DRDY 触发 + FIFO 水位
mpu6050_events.c	运动检测阈值事件
mpu6050_selftest.c	factory self-test(sysfs)
Makefile / Kconfig	双用途构建 + 在树配置

5.2 关键设计点

- **多变体 chip_info 表**:{name, whoami, has_fifo},probe 时按 WHO_AM_I 校验,支持 6050/6500/9250。
- **regmap**:统一寄存器访问,自带缓存与 debugfs/regmap/*/registers ,换总线零改动。
- **通道**:accel x/y/z、 gyro x/y/z、 temp + 软件时间戳;RAW 分离、 SCALE 按类型共享、 SAMP_FREQ 全局共享;并提供 *_available 列表。
- **量程换算**:accel/gyro 四档全量程对应 SI 每-LSB 值,以 IIO_VAL_INT_PLUS_NANO 表达。
- **采样率**:通过 SMPLRT_DIV,rate = 1000/(1+div),校验合法档位。
- **触发**:自建 data-ready 触发,set_trigger_state 使能片上 DATA_RDY/FIFO_OFLOW 中断。
- **事件**:accel 运动检测,MOT_THR 阈值 + INT_ENABLE 使能,标准 IIO_EV_TYPE_MAG/RISING。
- **自检**:每轴测 16 次均值,自检开/关之差落在窗口内即 pass,读后恢复正常配置。

5.3 与 V4L2 的概念对照

概念	V4L2	本 IIO 驱动
能力描述	VIDIOC_ENUM_FMT / v4l2_format	iio_chan_spec[] + *_available
能力协商	协商 format/分辨率	选 scan_elements + scale/ODR
缓冲队列	vb2_queue(videobuf2)	iio_buffer(kfifo)
硬件触发搬运	frame-done IRQ → dqbuf	DRDY/FIFO IRQ → pollfunc push
用户取数据	VIDIOC_DQBUF / read()	read(/dev/iio:deviceN)
多前端	subdev + media controller	core + i2c/spi 前端拆分

5.4 应用场景

场景	用到的能力
手机/平板 屏幕旋转、计步、姿态	accel/gyro 数据流 + 低功耗 ODR
可穿戴/TWS 佩戴与敲击检测	运动检测事件、wake-on-motion
无人机/平衡车/机器人 姿态解算	高采样率 + DRDY 硬件触发 + 精确时间戳
AR/VR 头显 头部追踪	高吞吐缓冲流 + FIFO 水位降 CPU 负载
IoT 跌落/震动/防拆告警	运动阈值事件、超低功耗待机
工业设备 振动监测	连续缓冲采集 + libiio 上位机分析
always-on sensor hub	FIFO 批量 + runtime PM,主 CPU 尽量休眠

5.5 现状与已知边界(诚实说明)

- 代码/文档/CI/测试已完成并推送;硬件 bring-up 按 M1→M4 推进中。
 - 内核 API 基线 $\geq 5.15/6.1$;STM32MP157 的 ST 5.4 BSP 需 README 中的兼容 shim。
 - self-test 采用整数响应窗口判定;浮点 factory-trim 为后续扩展。
 - FIFO 水位路径当前对整批帧共用一个时间戳,逐帧时间插值为可选增强。
- 本项目为学习 / 作品集用途,非 Invensense 官方驱动。仓库:github.com/fyy0619-cell/xiangmu